AZURE DAY TORINO 2026

MCP server, A2A, Skill, mamma mia che mal di testAI!



Andrea Tosato



Microsoft

msc TECHNOLOGY

TORINO.NET

NET CODE

AZURE DAY

AZURE DAY

Platinum Sponsor

Microsoft

msc TECHNOLOGY

Technical Sponsor

TORINO .NET

NET CODE

JET BRAINS



Tre domande prima di aggiungere componenti

MCP

Quale strumento posso usare?

Scoperta e invocazione di operazioni con un **contratto comune**.

A2A

A chi posso delegare il task?

Un altro agente mantiene **logica**, **tool** e **contesto** propri.

SKILL

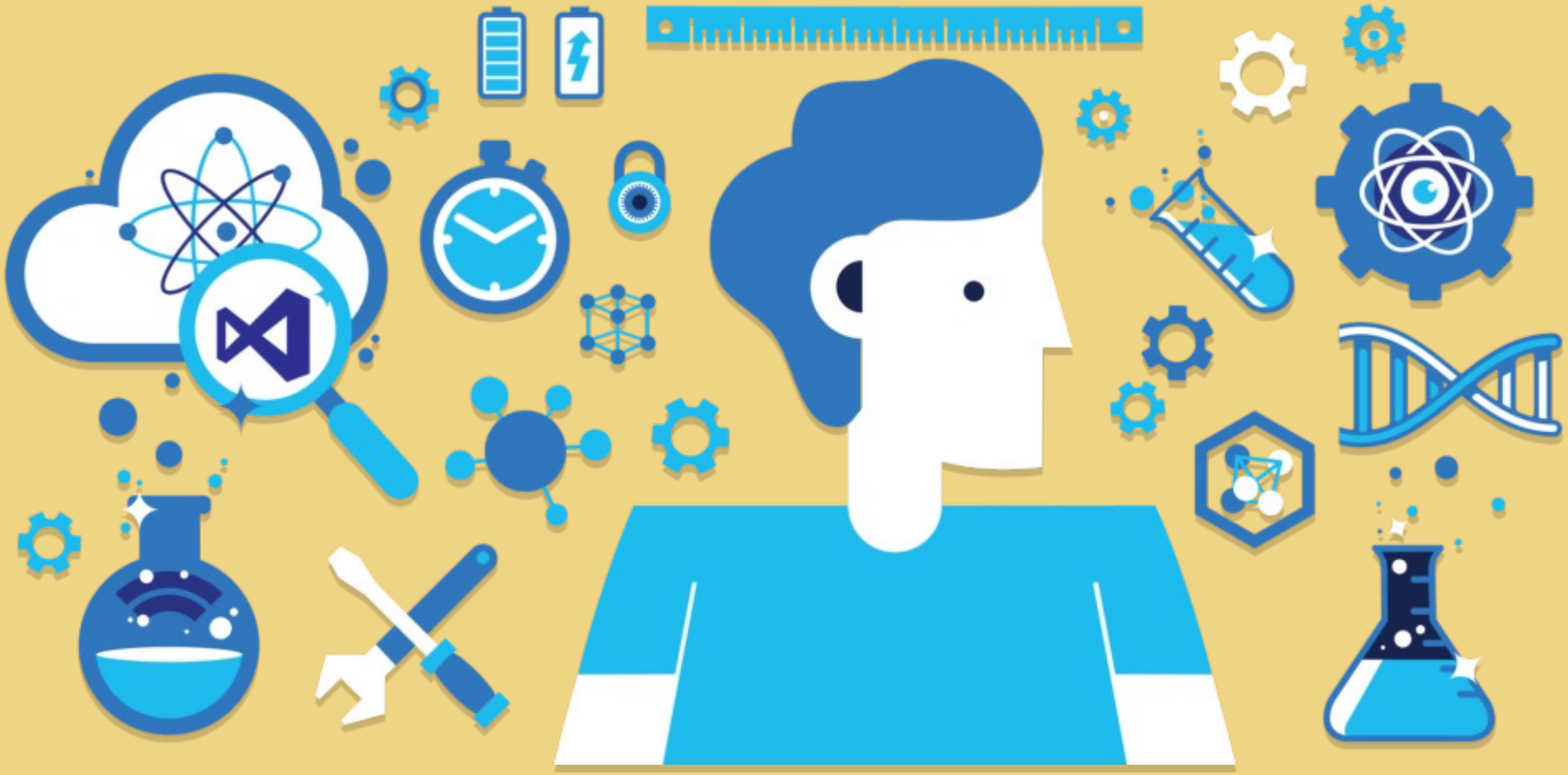
Come svolgo questo compito?

Istruzioni e **risorse riutilizzabili**, *caricate quando servono*.

Tool ≠ agente autonomo. Procedura ≠ protocollo di rete.

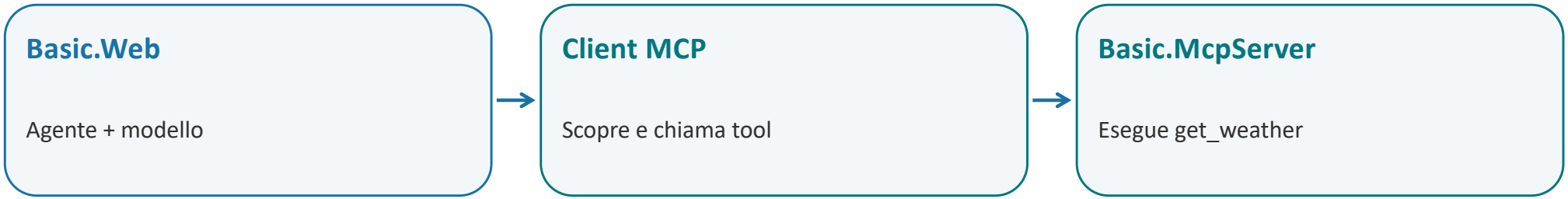
AZURE DAY

Demo





MCP: l’agente usa uno strumento



Richiesta → tool → dato → risposta dell’agente locale

- Il modello decide quando usare il tool.
- Il tool restituisce dati, non un secondo ragionamento agentico.
- MCP separa il contratto operativo dall’implementazione.



MCP: quando usarlo, quando basta una funzione

COS'È

Protocollo per esporre tool, risorse e prompt a client compatibili.

Non è un agente autonomo.

USALO SE

Lo stesso catalogo serve più agenti o applicazioni.

Vuoi separare deploy e integrazioni.

NON SERVE SE

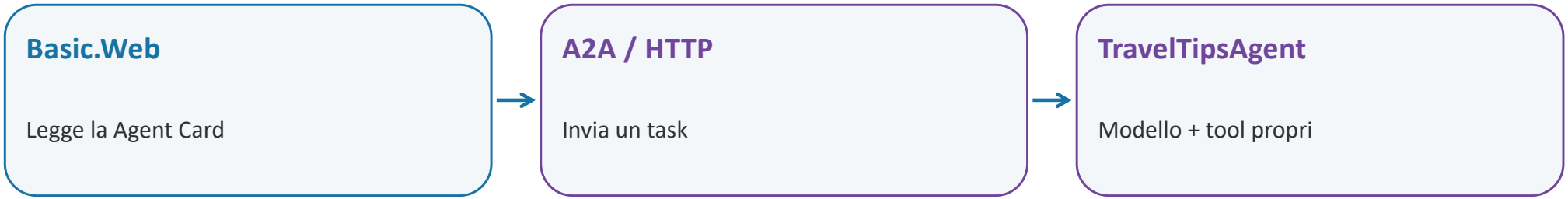
Una funzione locale è sufficiente.

Il trasporto remoto non aggiunge riuso o separazione.

Esempio: get_weather(city) restituisce dati. La risposta la costruisce l'agente.



A2A: delego un task a uno specialista



Il risultato torna al Web, che lo presenta all'utente.

- La Agent Card descrive identità, capacità ed endpoint.
- Lo specialista esegue il proprio ragionamento.
- Nella demo: modello sia nel processo remoto sia nel Web.



A2A: quando delegare un risultato, non un tool

COS'È

Protocollo tra agenti per affidare task e ricevere risultati.

La Agent Card rende note le capacità.

USALO SE

Lo specialista ha ownership, strumenti e ragionamento propri.

Vuoi delegare un obiettivo.

VALUTA IL PREZZO

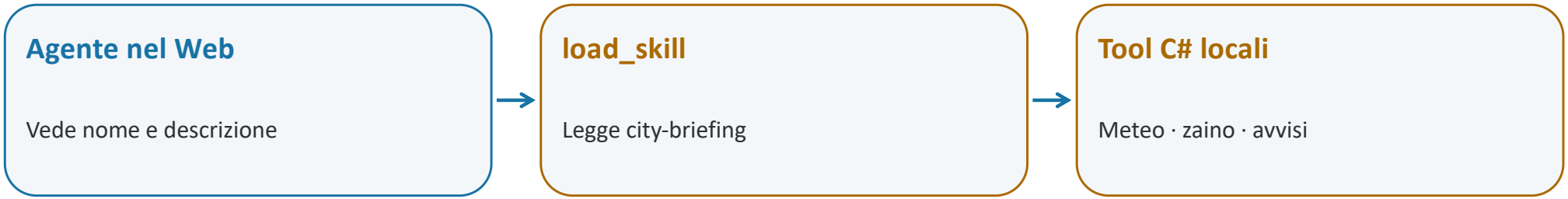
Un altro prompt e altre chiamate al modello.

Più rete, latenza e stato da governare.

Esempio: trova un opzione di consegna compatibile con la deadline.



Skill: una procedura per lo stesso agente



SKILL.md guida il lavoro: non crea un altro agente.

- Istruzioni e risorse sono caricate quando servono.
- I tool vanno comunque resi disponibili all’agente.
- Qui non serve un processo «Skill server».



Skill: quando il sapere è una procedura

COS'È

Istruzioni e risorse riutilizzabili per svolgere un compito.

SKILL.md descrive come procedere.

USALA SE

Vuoi una procedura coerente senza creare un altro agente.

Vuoi caricarla solo quando serve.

NON CONFONDERLA

Non crea un server.
Non esegue da sola i tool.
Non isola il ragionamento in un altro processo.

Esempio: city-briefing guida meteo, zaino e precauzioni nello stesso agente.

Specification - Agent Skills

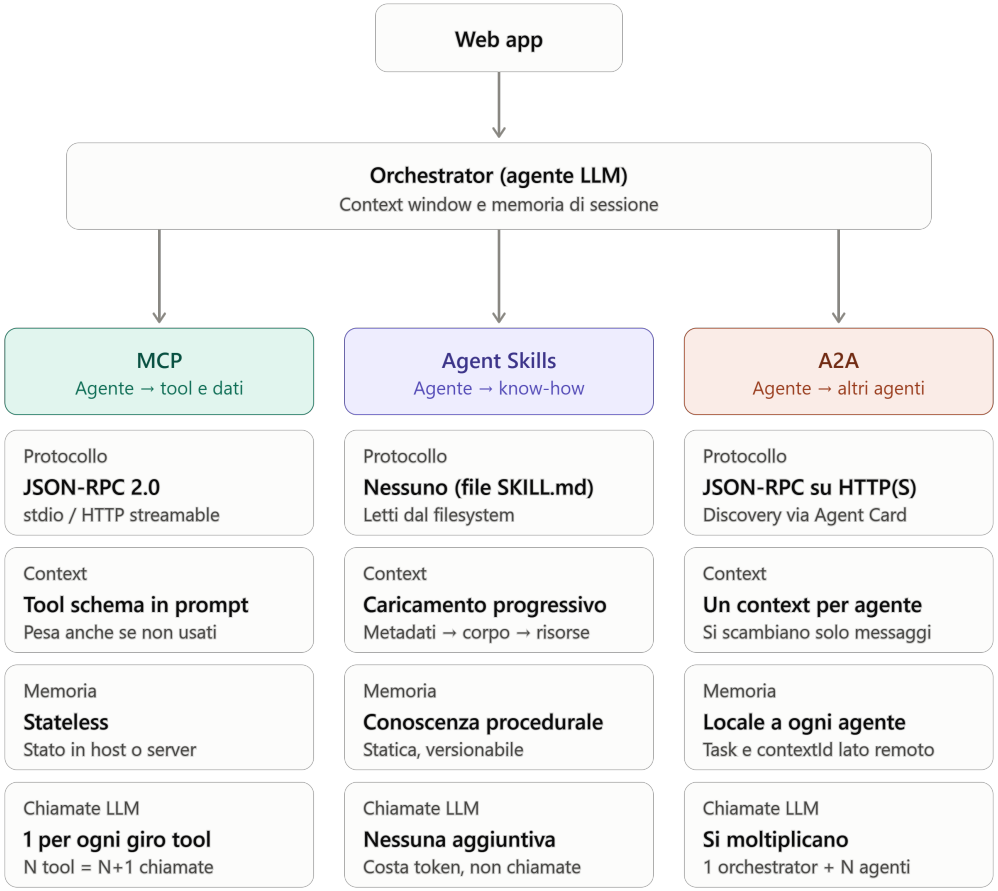


Non sono alternative equivalenti

ASPETTO	MCP	A2A	SKILL
Contratto	Operazioni / dati	Capacità di un agente	Istruzioni / risorse
Modello in BASIC	Nel Web	Nel remoto + nel Web	Nel Web
Confine in BASIC	Server tool via HTTP	Agente via HTTP	Libreria + file locali
Responsabilità	Eseguire il tool	Svolgere il sottotask	Guidare la procedura
Può combinarsi?	Con Skill e A2A	Con Skill e MCP	Con MCP e A2A

Sono complementari: scegli quale responsabilità distribuire.

Riassumendo



AZURE DAY

Microsoft Agent Framework



Perché Microsoft Agent Framework?

MENO PLUMBING

Un runtime per modello e tool.

Sessioni e context provider.

Streaming e cancellazione.

CONTROLLO

Middleware e telemetria.

Workflow espliciti quando servono.

Policy e approvazioni applicative.

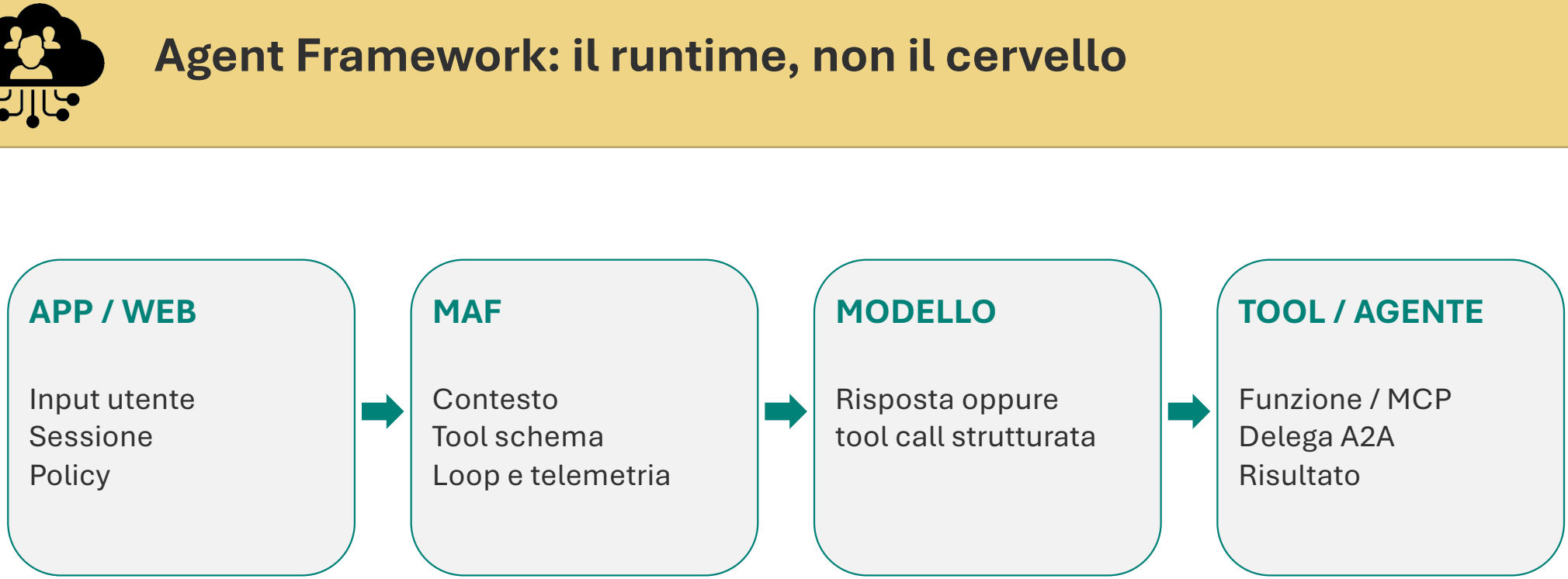
INTEGRAZIONI

Client dei modelli.
Funzioni C# e MCP.
Agenti remoti e Skill.

Capacità variabili per provider.

È utile per governare il ciclo agentico. Non serve un framework per ogni chiamata LLM.

Agent Framework: il runtime, non il cervello



Se c'è una tool call: risultato -> nuovo input al modello -> eventuale risposta finale.

Il modello propone una chiamata. Il codice controlla ed esegue.



AZURE DAY

PROMPT



Il prompt agentico non è una sola stringa

ISTRUZIONI	Ruolo, obiettivo, vincoli, policy, procedura e criterio di stop.
CONTESTO E DATI	Fatti pertinenti, documenti recuperati, Skill caricata, risultati dei tool.
CONVERSAZIONE	Domanda corrente, history selezionata, tool call e relativi risultati.
TOOL E FORMATO	Nomi, descrizioni, parametri / JSON Schema, eventuale schema di risposta.

Input al modello = contenuto + struttura. Dati esterni non sono istruzioni fidate.



Dove si compone il prompt in Agent Framework

Instructions	ChatOptions.Instructions -> regole e comportamento dell'agente.
Run / sessione	RunAsync(input, session) -> domanda + history, se la sessione e riutilizzata.
Context provider	AIContextProviders -> contesto pertinente; AgentSkillsProvider -> Skill.
Tools + client	ChatOptions.Tools -> schemi; IChatClient -> richiesta al provider.

Il prompt finale si verifica nella richiesta al modello, non solo nel file di istruzioni.



Token: quali contatori stiamo leggendo?

INPUT

Tutto ciò che entra.

Prompt, history, tool schema e risultati.

Token != parole.

CACHE

Read: parte di input riusata.
Write: nuova cache scritta, se esposta.

Non sommare i read all input.

OUTPUT

Risposta + chiamate tool.
Reasoning, se previsto dal modello.

Non tutto è visibile all'utente.

Leggi lo schema usage del provider: le categorie non sono sempre disgiunte.



Stabile davanti, variabile dopo: progettare il prefisso

PREFISSO STABILE

Regole versionate + schema tool stabile
Procedura / riferimenti comuni pertinenti
Stesso contenuto, stesso ordinamento

CODA VARIABILE

Richiesta utente
Dati recuperati / tool
Timestamp e ID

Da evitare: request ID o data corrente prima di tutte le regole comuni.

Stabilità non significa contesto inutile: prima seleziona, poi rendi riusabile.



Quanto risparmi? Conta anche l'output.

TARIFFE IPOTETICHE

Per 1 milione di token:
Input: $2
Cache read: $0,20
Output: $8

Cache write = 0.

SENZA CACHE

10.000 input -> $0,020
1.000 output -> $0,008

Totale: $0,028

CON 80% INPUT CACHED

2.000 nuovi -> $0,004
8.000 read -> $0,0016
1.000 output -> $0,008

Totale: $0,0136

Risparmio totale: 51,4%. Aggiungi eventuali costi di cache write.



MAF non garantisce la cache: governa la richiesta

NEL FRAMEWORK

Istruzioni versionate.
Context provider mirati.
Tool schema stabili.
History selezionata.

NEL PROVIDER

Modello / deployment.
Cache key e breakpoint se supportati.
Retention e policy.

Non sono opzioni universali.

NELLA TELEMETRIA

Input / output.
Cache read / write.
Chat e tool call.
Costi, latenza ed esito.

Misura il payload effettivo.

BASIC non dimostra un cache hit: verifica soglia, provider e contatori.



Un prompt utile è un contratto osservabile

PROMPT DESIGN

Riduci ambiguità prima di ridurre token.

1 **RISULTATO**

Cosa deve essere vero alla fine?

2 **CONTESTO**

Quali fatti servono davvero?

3 **VINCOLI**

Policy, sicurezza, consenso, budget

4 **PROVE**

Fonti, tool, dati da citare

5 **STOP**

Quando completare o chiedere chiarimenti

6 **FORMATO**

Struttura e livello di dettaglio

Tagliare prima di chiarire = risparmiare token per ottenere più retry.



Ottimizzare il prompt: prima e dopo

PRIMA: VAGO E RIDONDANTE

Sei un assistente molto esperto.
Aiuta sempre al meglio.
Sii accurato e completo.
Ripeti tutte le informazioni.
Usa strumenti se utile.

DOPO: CONTRATTO VERIFICABILE

Demo: solo Torino, dati sintetici.
Meteo: usa get_weather.
Se manca la città, chiedila.
Non inventare fatti o recuperi.
Risposta breve: condizioni e gradi.

Ottimizzare = meno ambiguità e lavoro inutile, non meno vincoli necessari.



History e risultati dei tool: il costo cresce a ogni giro

SELEZIONA

Solo tool pertinenti.
RAG mirato.
Risultati con campi utili.

Non incollare payload enormi.

COMPRIMI CON CRITERIO

Preserva fatti e vincoli.
Conserva call / result.
Valuta costo del riassunto.

Non mescolare le sessioni.

FERMA IL LOOP

Budget di chiamate e output.
Timeout / cancellazione.
Retry solo quando utili.

Stop verificato nel codice.

Riduci input ripetuti, giri inutili e output superfluo. Poi verifica la qualità.



Tre scelte, un metodo di ottimizzazione

SCEGLI IL CONFINE

MCP: operazione.
A2A: responsabilità.
Skill: procedura.

Combinabili, non equivalenti.

COMPONI IL CONTESTO

Instructions + history.
Tool schema + contesto.
Skill quando pertinente.

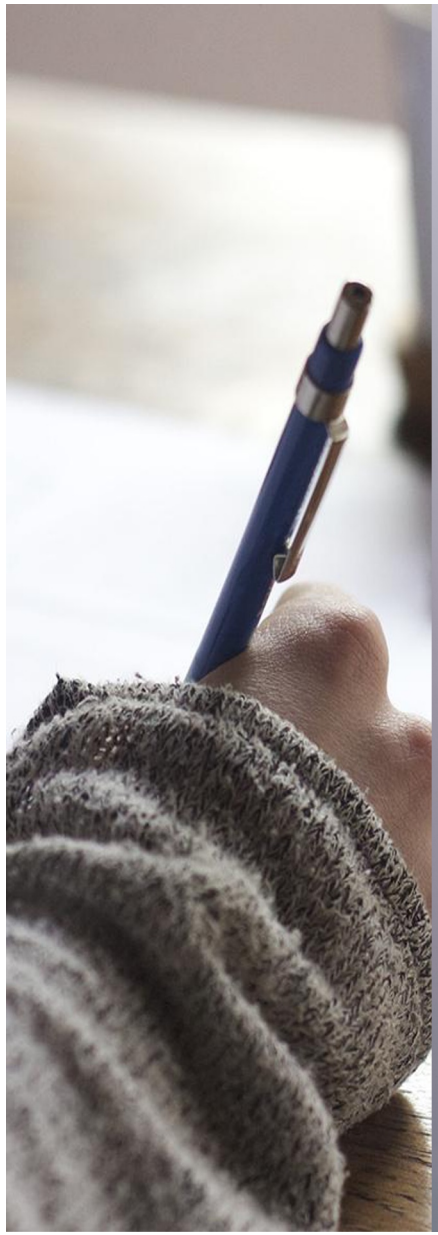
MAF governa il ciclo.

MISURA IL RISULTATO

Qualità e sicurezza.
Input / output / cache.
Latenza e numero di chat.

Costo per esito corretto.

Il sistema migliore non è quello con più agenti: è quello che svolge bene il lavoro utile.



Questions & Discussion



https://github.com/andreatosato/AzureDayTorino2026

AZURE DAY



Vote for this session!!

https://vote.dotnetdev.it/vote/l44tzzta/1304496